Working with Buttons

ECE 2564, Embedded Systems

Fall 2025

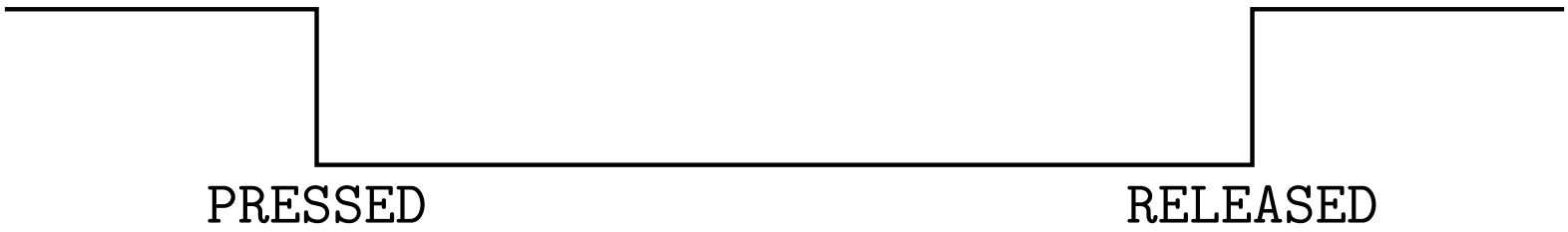
Working with Buttons

- *Button tapping* is the function when we `PRESS` a button and then when we `RELEASE` it
- Push-button tapping
  - Behavior
  - FSM
  - Code

Debouncing of Switches

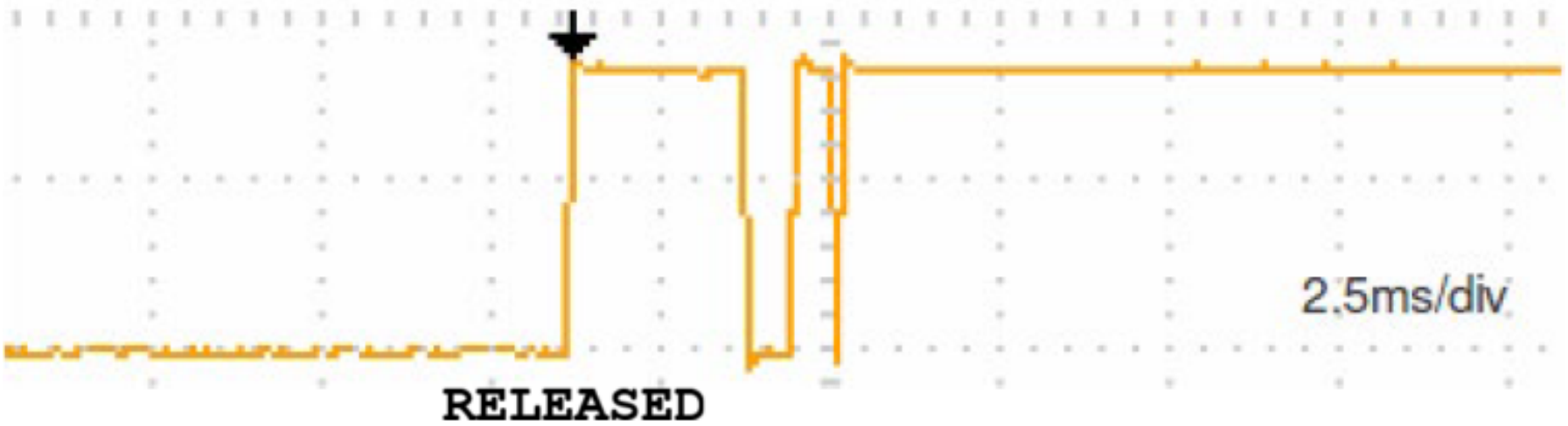
- When you operate a physical switch the parts that make electrical contact tend to "bounce" against one another
- "Contact bounce" cause problems for system developers

An Ideal Switch[1]

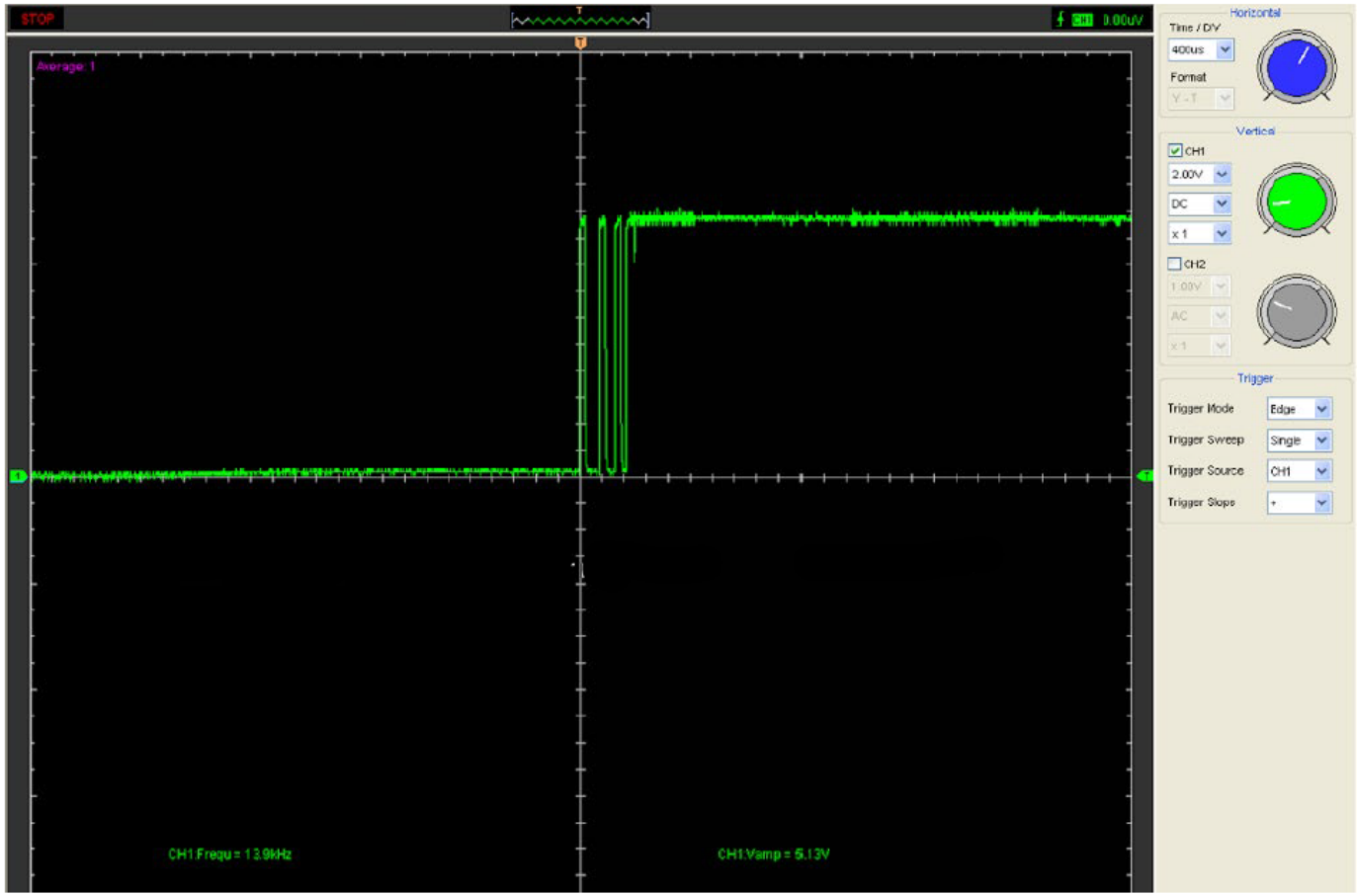


[1] With pullup resistor, `PRESSED = 0`, `RELEASED = 1`

Oscilloscope Trace from an Actual Mechanical Switch



Another Oscilloscope Trace from an Actual Mechanical Switch



Why is Contact Bounce a Problem?

- Very often the reason to press a switch is to initiate some action
- The *transition* of the switch needs to be detected, but bouncing causes many spurious transitions

- Do we want to initiate some action after every (noisy) transition?
  - No!

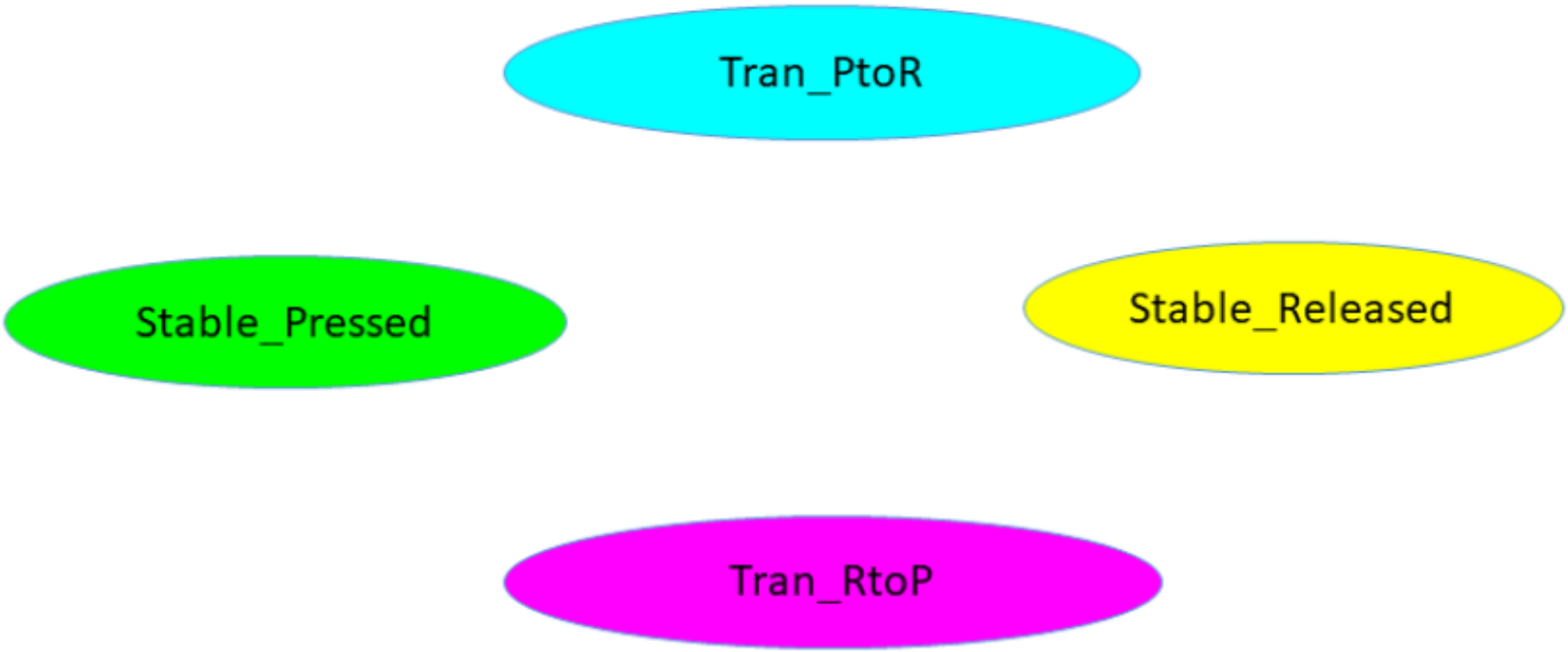
Solution Approaches

- Hardware solutions
  - Analog circuit: Place a low-pass analog filter (resistor/capacitor) between the switch and the microcontroller
  - Digital circuit: Use a shift register and wait for the signal to stabilize
- Software solutions
  - Implement delays in the code that wait for the signal to stabilize
- Hybrid solutions
  - Use software along with a hardware timer module on the microcontroller to wait for the signal to stablize

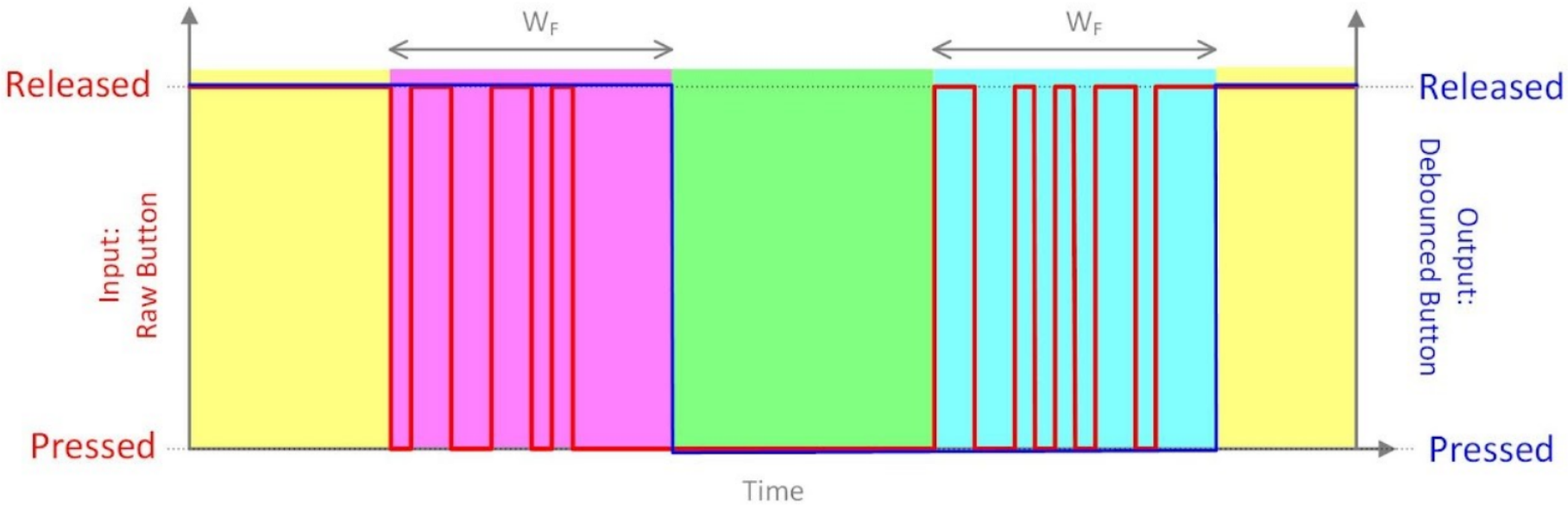
How to Achieve Button Debouncing with a Hybrid Solution

- Use a FSM
  - Software implementation
  - Assume that a *timer* is available (to be discussed later in the course)
- There are two "stable" states for the FSM
  - `PRESSED`
  - `RELEASED`
- There are two "transient" states for the FSM
  - Transient_Pressed_to_Released
  - Transient_Released_to_Pressed

The States of the Button FSM

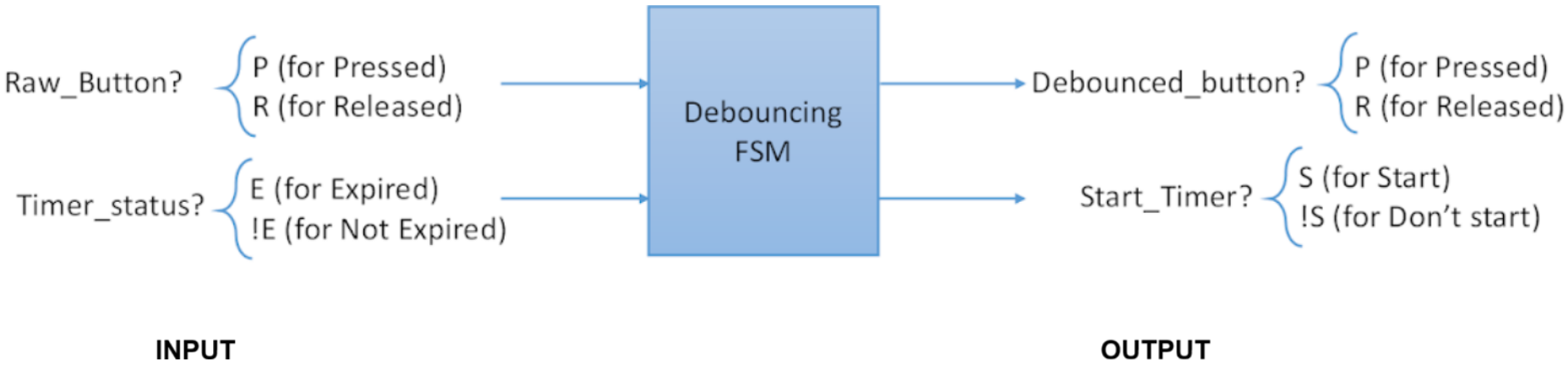


Transition Timeline

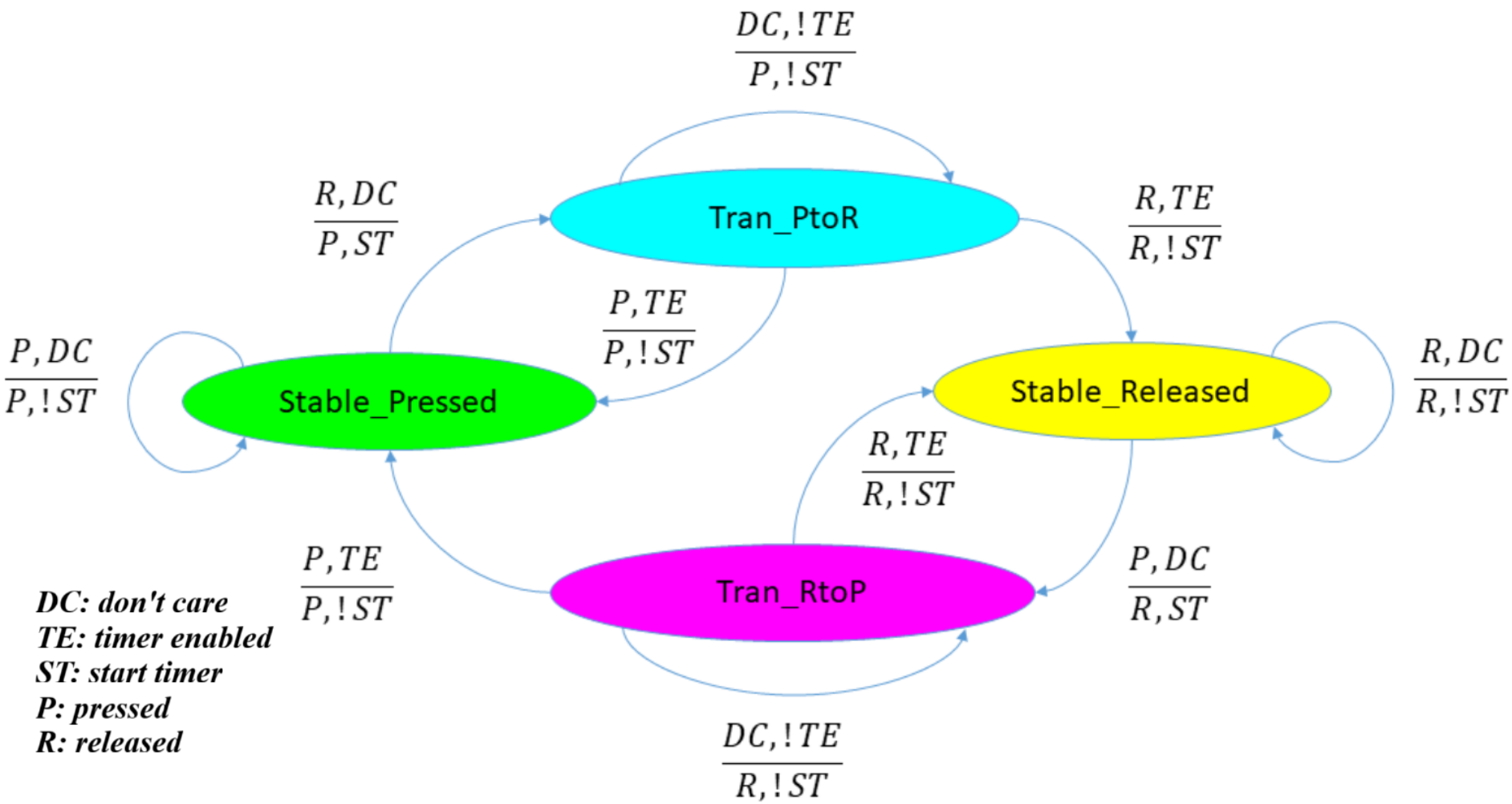


- When is a transition is sensed, we begin a waiting period ($\sim 100\,\text{ms}$)
- At the end of the waiting period, if the transition has actually happened we move to the new stable state

Input/Output of the FSM



State Transition Diagram for the FSM



Modular Design

- *Modular design* is the key to good software engineering and successful software development team projects
- We must decompose code into modules

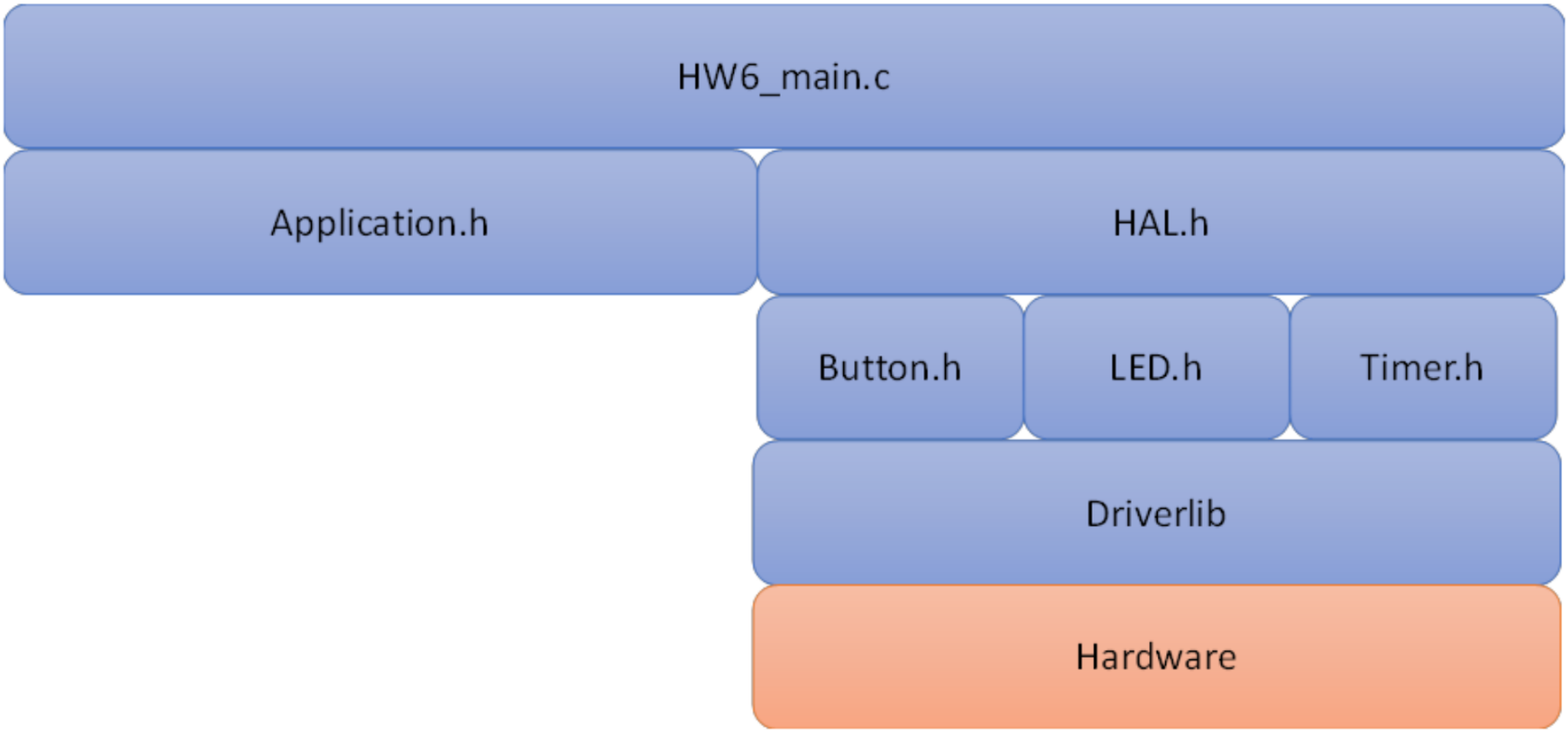
Motivation

- Efficient code development
  - Smaller sub-programs are
    - Easier to understand
    - Faster to develop
    - Easier to test
    - Easier to maintain, modify and improve
- More potential for code re-use
- Better teamwork and collaboration

Modular Code

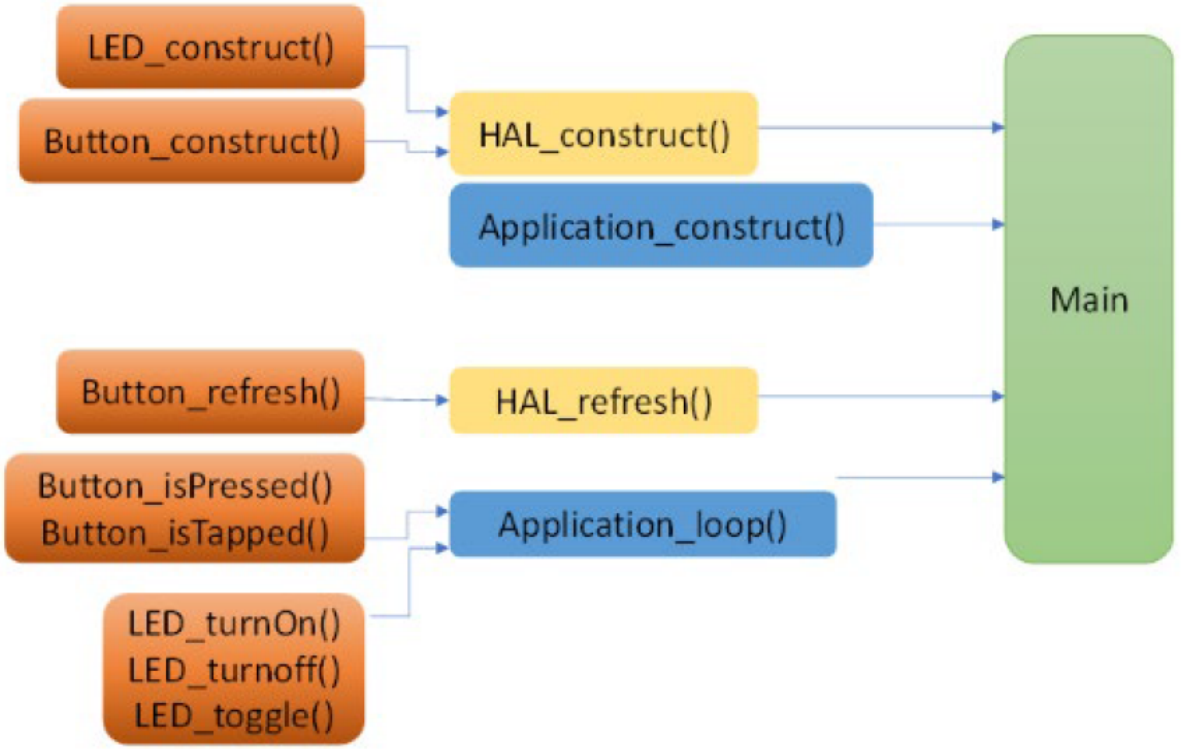
- In C, and in most programming languages, a single project can consist of multiple files:
  - *.h files contain function declarations, macros, structs and enums needed by other files
    - Should describe the interface for others to use
  - *.c files contain function definitions, as well as everything else that is internal to that file and not needed by other files
- In a well-written project reading the *.h files and understanding the overall structure of code is enough to use the functions without understanding the inner workings of the functions in the *.c files
  - This is the promise of a well-developed API

Typical Structure of Project in this Course

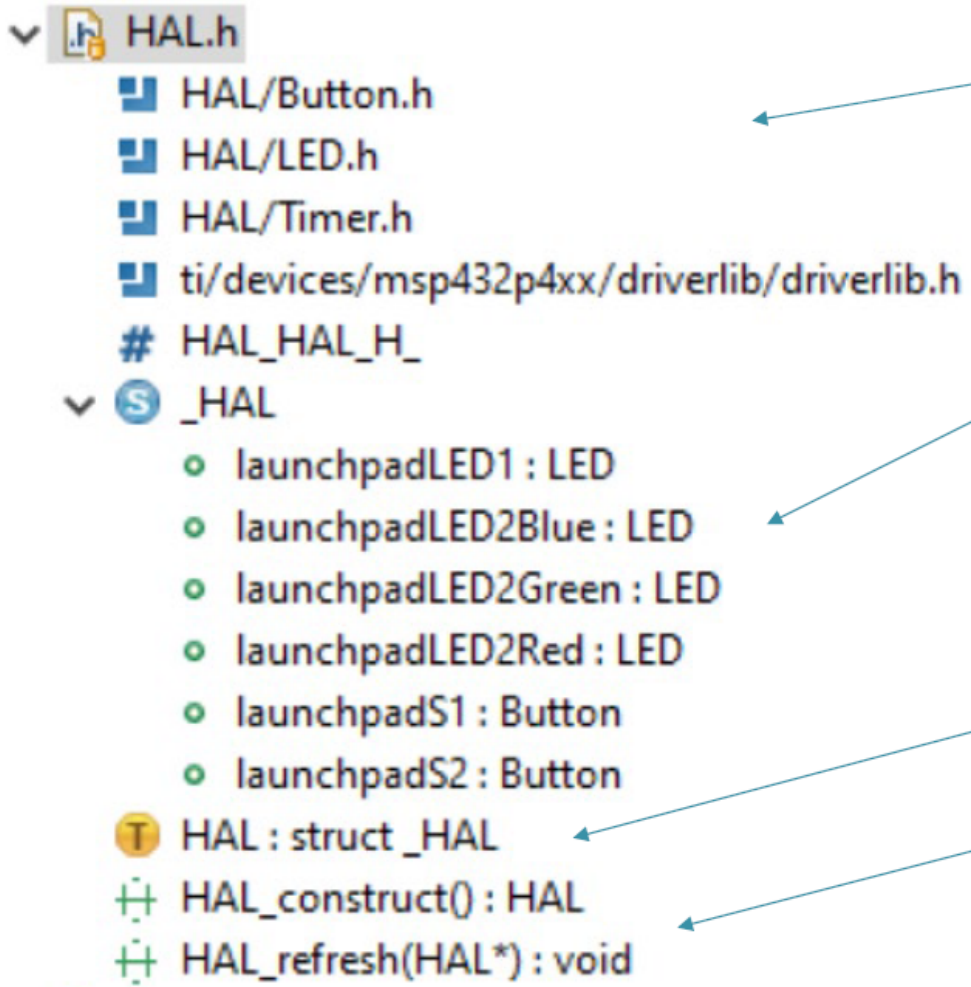


Modularity of the Projects

- Application
  - All application-specific variables reside here
- HAL
  - All hardware-specific variables reside here
- Initialize Application and HAL separately



File Explorer View in CCS



- Files
- Structures
- Typedef
- Functions

Separation of Code

- Make sure the hardware components (buttons, LEDs, etc.) that you want to use are defined in the HAL
- For good separation of code into (1) application and (2) HAL
  - Need to define the HAL constructs and functions
  - What's in the following `struct`? Observe there are 7 LEDs
  - You will need to add more LEDs/buttons if needed

```
26 struct _HAL
27 {
28     // LEDs - Left Launchpad LED (the LED is only a single color - red)
29     LED launchpadLED1;
30
31     // LEDs - Right Launchpad LEDs (one LED struct for red, one for blue, etc.)
32     LED launchpadLED2Red;
33     LED launchpadLED2Blue;
34     LED launchpadLED2Green;
35
36     LED boosterPackRed;
37     LED boosterPackBlue;
38     LED boosterPackGreen;
39
40     // Buttons - Launchpad S1 and S2
41     Button launchpadS1;
42     Button launchpadS2;
43 };
```

HAL "constructor"

- Construct and initialize LEDs and Buttons
- How would you construct the boosterPack LEDs and buttons

```
16 HAL HAL_construct()
17 {
18     // The API object which will be returned at the end of construction
19     HAL hal;
20
21     // Initialize all LEDs by calling their constructors with correctly-defined arguments.
22     hal.launchpadLED1 = LED_construct(LAUNCHPAD_LED1_PORT, LAUNCHPAD_LED1_PIN);
23
24     hal.launchpadLED2Red   = LED_construct(LAUNCHPAD_LED2_RED_PORT  , LAUNCHPAD_LED2_RED_PIN  );
25     hal.launchpadLED2Green = LED_construct(LAUNCHPAD_LED2_GREEN_PORT, LAUNCHPAD_LED2_GREEN_PIN);
26     hal.launchpadLED2Blue  = LED_construct(LAUNCHPAD_LED2_BLUE_PORT , LAUNCHPAD_LED2_BLUE_PIN );
27
28     // Initialize all Buttons by calling their constructors with correctly-defined arguments.
29     hal.launchpadS1 = Button_construct(LAUNCHPAD_S1_PORT, LAUNCHPAD_S1_PIN);        // Launchpad S1
30     hal.launchpadS2 = Button_construct(LAUNCHPAD_S2_PORT, LAUNCHPAD_S2_PIN);        // Launchpad S2
31
32     // Once we have finished building the API, return the completed struct.
33     return hal;
34 }
```

Example HAL Function

- What does this function do?
- What are the functions arguments?

```
void turnLaunchpadRGB(HAL* hal_p, _RGBstatus RGBstatus) {
    switch(RGBstatus)
    {
    case OFF:
        LED_turnOff(&hal_p->launchpadLED2Blue);
        LED_turnOff(&hal_p->launchpadLED2Red);
        LED_turnOff(&hal_p->launchpadLED2Green);
        break;
    case GREEN:
        LED_turnOff(&hal_p->launchpadLED2Blue);
        LED_turnOff(&hal_p->launchpadLED2Red);
        LED_turnOn(&hal_p->launchpadLED2Green);
        break;
    case BLUE:
        LED_turnOn(&hal_p->launchpadLED2Blue);
        LED_turnOff(&hal_p->launchpadLED2Red);
        LED_turnOff(&hal_p->launchpadLED2Green);
        break;
    case RED:
        LED_turnOff(&hal_p->launchpadLED2Blue);
        LED_turnOn(&hal_p->launchpadLED2Red);
        LED_turnOff(&hal_p->launchpadLED2Green);
    }
}
```

How to Add More Colors?

- Suppose you want to add yellow: RED + GREEN

```
case RED:
    LED_turnOff(&hal_p->launchpadLED2Blue);
    LED_turnOn(&hal_p->launchpadLED2Red);
    LED_turnOff(&hal_p->launchpadLED2Green);
    break;
case CYAN:
    break;
case YELLOW:
    break;
case MAGENTA:
    break;
case WHITE:
    break;
```

HAL_refresh

- Refresh buttons to reset their debounced state (provided for you)

```
void HAL_refresh(HAL* hal_p)
{
    // Refresh Launchpad buttons
    Button_refresh(&hal_p->launchpadS1);
    Button_refresh(&hal_p->launchpadS2);

}
```

Application Constructor

- Simply initialize the FSM state

```
Application Application_construct()
{
    Application app;

    app.smallFSMstate = Green;
    return app;
}
```

Example Application

- Button is defined in HAL
- LED is also defined in HAL
- To check if a button is pressed and turn on the LED:

```
void Application_loop(Application* app_p, HAL* hal_p)
{
    // Turn on the LED1 on Launchpad on whenever the Launchpad S1 is pressed
    // This is a simple logic which has no memory
    // Use the below snippet to test part A and B
    // Comment this snippet when done with part A and B
    if (Button_isPressed(&hal_p->launchpadS1))
    {
        LED_turnOn(&hal_p->launchpadLED1);
    }
    else
    {
        LED_turnOff(&hal_p->launchpadLED1);
    }
```

Example Application (continued)

- Tie HAL and application in `main()`
- Construct HAL and Application
- While loop: Refresh HAL

```
int main(void)
{
    // Stop Watchdog Timer - THIS SHOULD ALWAYS BE THE FIRST LINE OF YOUR MAIN
    WDT_A_holdTimer();

    // Initialize the system clock and background hardware timer, used to enable
    // software timers to time their measurements properly.
    InitSystemTiming();

    // Initialize the main Application object and HAL object
    HAL hal = HAL_construct();
    Application app = Application_construct();

    // Main super-loop! In a polling architecture, this function should call
    // your main FSM function over and over.
    while (true)
    {
        HAL_refresh(&hal);
        Application_loop(&app, &hal);
    }
}
```

HW6 Video

Watch the below playlist to learn about the code structure used throughout the course. This video also provides step-by-step instructions on how to complete HW6.

HW6 Video

Homework 6 Highlights

- Add new LEDs to the project
- Add new buttons to the project
- Observing how the tapping operation works
- Looking into debouncing
- Understanding the general structure of an embedded application

Summary

- Modular Programming
  - Why? How is it done technically?
  - Art of Programming
- Example with HAL and Application
  - Learn and master by doing (HW6)
  - Separation allows portability